

CSA_DATA_DIGGER Prototype

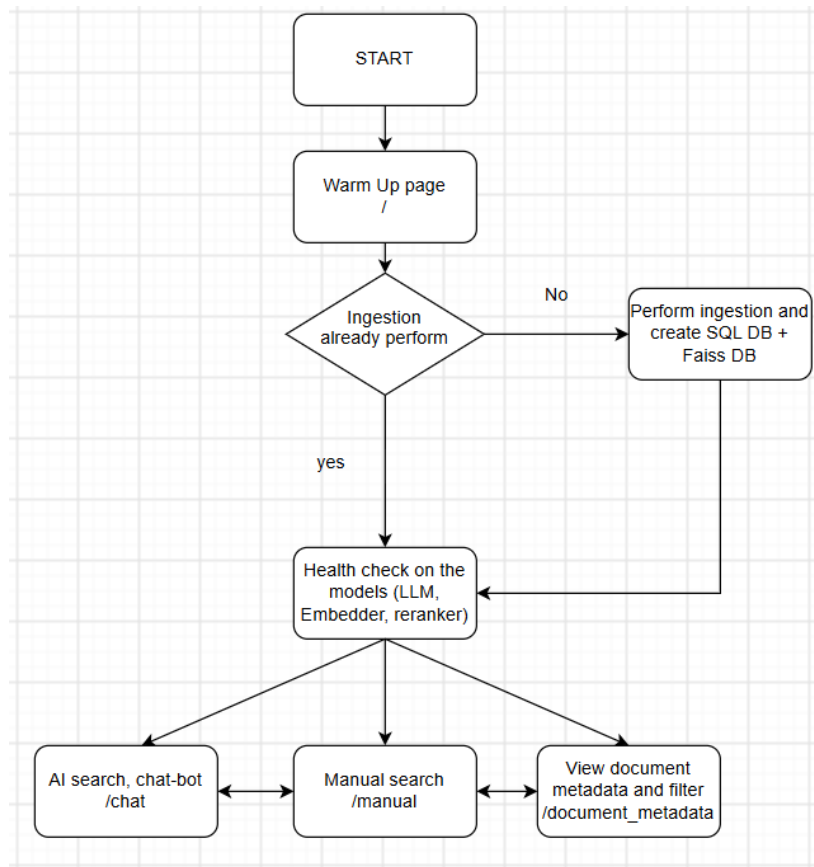
1. Overview of the prototype
2. Current Architecture and Workflow
3. Proposed Solution

1. Overview of the prototype

CSA_DATA_DIGGER is a very basic prototype developed to support document exploration and question answering within a secure environment. The application can run on both offline Linux machines and Windows environments and is deployed as a web application.

The system uses RAG architecture built entirely on local resources, including a local Large Language Model, embedding model, and reranking model. This approach allows the application to operate without internet connectivity while maintaining full control over sensitive data.

2. Current Architecture and Workflow



Document Ingestion and Warm-Up Process

During the first application startup, an ingestion process is executed.

- Scans all documents located under a configured root directory.
- Extracts document metadata.
- Split document content into text chunks. Generates vector embeddings for each chunk.
- Stores all embeddings in a FAISS vector database. Store the original text chunks and associated metadata in a SQLite database.

Once ingestion is complete, the system performs a warm-up sequence to ensure all required services are available. This includes health checks for local LLM, embedder, reranker. The application only becomes available to users once all services are confirmed operational.

Available Features

After the warm-up process is completed, users can access three main pages:

/chat

- The Chat page provides an AI-assisted question-answering interface.
- A similarity search is performed against the FAISS database using cosine similarity.
- The most relevant document chunks are retrieved. A reranking model evaluates and ranks the results. The top five chunks are passed to the LLM.
- Support for both English and French.
- Source attribution for every response with confidence level displayed alongside each answer.
- LLM_TEMPERATURE = 0.0 to maximize response consistency and reduce hallucinations.

/manual

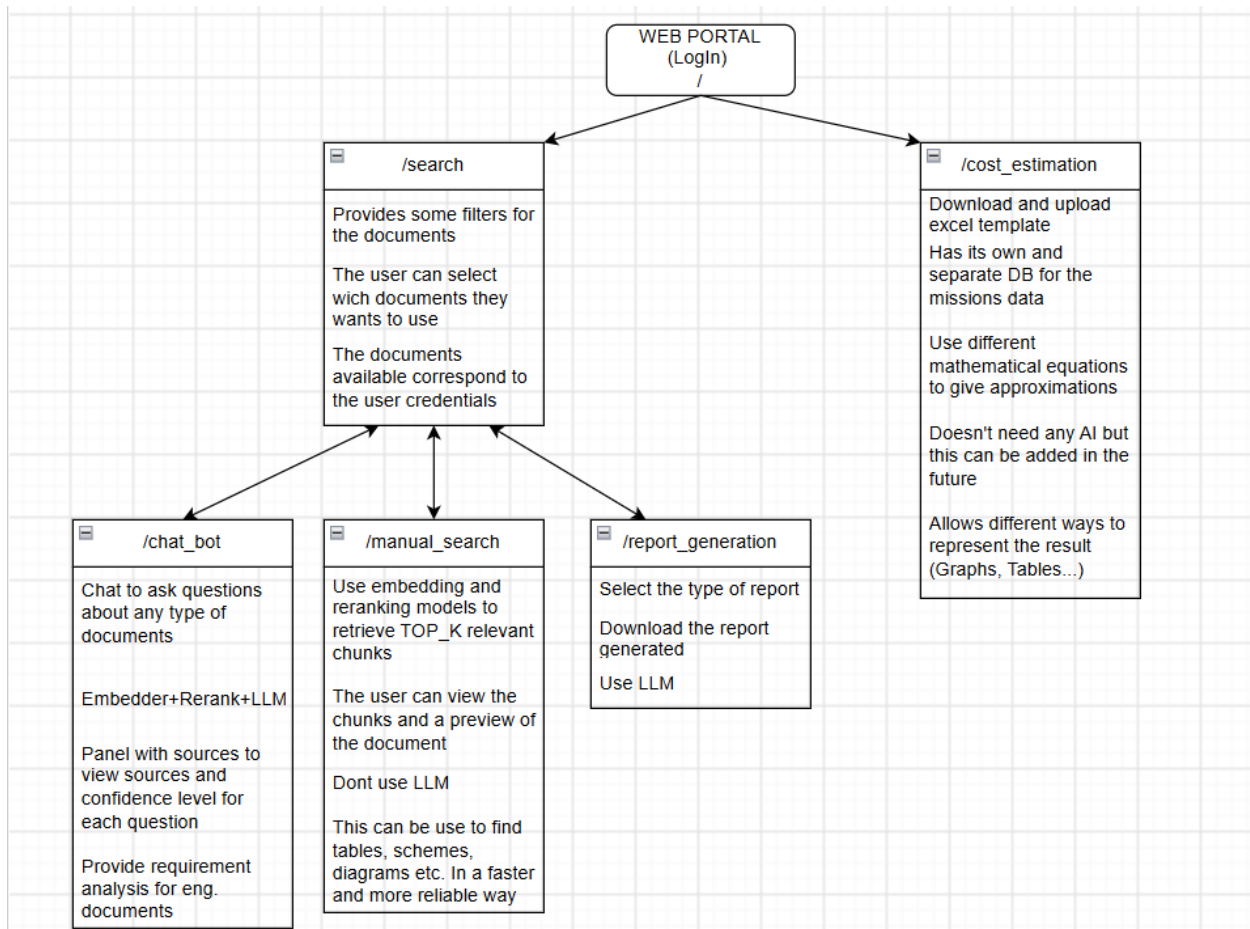
- Relevant chunks are retrieved using embeddings and reranking.
- The system displays an image of the corresponding document pages from which the chunks originated. Users can navigate through the search results using Next and Previous controls.
- Direct visual verification of source content. Reduced dependence on AI-generated interpretations.
- Ability to determine the veracity and accuracy of a chunk by verifying whether information originates from PDFs, emails, excel spreadsheet etc. Find tables, diagrams, schemas and visualize them.

/document_metadata

- The Document Metadata page enables users to browse and filter document Metadata.
- Available filters may include Mission, Subject area (Engineering, Costing, Operations, etc.), Document type and any additional metadata fields extracted during ingestion
- This feature helps users quickly identify relevant documents before performing searches or analyses.

3. Proposed Solution

The current CSA_DATA_DIGGER prototype already provides a solid foundation for secure document retrieval, source validation, and AI-assisted question answering through a local RAG architecture. The next logical development priorities are moving from local machine to AZURE; this includes replacing local models with more performant Azure models and moving from local DB to use Azure servers and DB. Then authentication and authorization, followed by dedicated cost-estimation, requirements-analysis, and report-generation capabilities.



Authentication and authorization (Highest Priority)

The most important missing capability is user authentication and authorization. A login page should be added to the application so that user access can be restricted based on credentials and permissions. This functionality should ideally be integrated into the application entry point (/) so before users can access any content or it can have its own page as "/login" if the entry point "/" is reserved for other activities.

Mission Cost Estimation Module (/cost_estimation)

This page would:

- Use mathematical models rather than LLM reasoning.
- Compute mission cost estimates. Display graphs and forecast results.
- Operate from a dedicated cost database.

I do not believe an LLM should be responsible for performing cost calculations or generating authoritative cost analyses, as numerical accuracy is critical. In the future we may consider using LLM to explain results or summarize findings. All calculations should be performed by deterministic mathematical algorithms and validated cost models.

Requirements Analysis under “/search/chat_bot”

Requirements analysis could be integrated directly into the existing /chat page. A possible workflow would be:

1. User selects a set of requirements documents using available filters.
2. The system analyzes only the selected documents.
3. The application identifies: Missing requirements, potential inconsistencies, duplicates...

Report Generation (/search/generate_report)

A dedicated report-generation page could be implemented. Proposed workflow:

1. User selects one or more documents from the repository.
2. User chooses the report type.
3. The application gathers relevant information from the selected documents.
4. A predefined template is applied.
5. The final report is generated and made available for download.